

EXPERIMENT 1: Naïve Bayes Classifier with Different Feature Sets

```
import numpy as np
import pandas as pd

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score

from sklearn.naive_bayes import GaussianNB
from sklearn.naive_bayes import MultinomialNB
from sklearn.preprocessing import MinMaxScaler
```

```
data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

print(X.head())
print("Shape:", X.shape)
```

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness \
0      17.99      10.38      122.80      1001.0      0.11840
1      20.57      17.77      132.90      1326.0      0.08474
2      19.69      21.25      130.00      1203.0      0.10960
3      11.42      20.38       77.58       386.1      0.14250
4      20.29      14.34      135.10      1297.0      0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry \
0      0.27760      0.3001      0.14710      0.2419
1      0.07864      0.0869      0.07017      0.1812
2      0.15990      0.1974      0.12790      0.2069
3      0.28390      0.2414      0.10520      0.2597
4      0.13280      0.1980      0.10430      0.1809

   mean fractal dimension  ...  worst radius  worst texture  worst perimeter \
0      0.07871  ...      25.38      17.33      184.60
1      0.05667  ...      24.99      23.41      158.80
2      0.05999  ...      23.57      25.53      152.50
3      0.09744  ...      14.91      26.50       98.87
4      0.05883  ...      22.54      16.67      152.20

   worst area  worst smoothness  worst compactness  worst concavity \
0      2019.0      0.1622      0.6656      0.7119
1      1956.0      0.1238      0.1866      0.2416
2      1709.0      0.1444      0.4245      0.4504
3       567.7      0.2098      0.8663      0.6869
4      1575.0      0.1374      0.2050      0.4000

   worst concave points  worst symmetry  worst fractal dimension
0      0.2654      0.4601      0.11890
1      0.1860      0.2750      0.08902
2      0.2430      0.3613      0.08758
3      0.2575      0.6638      0.17300
4      0.1625      0.2364      0.07678

[5 rows x 30 columns]
Shape: (569, 30)
```

```
X1 = X.iloc[:, :10]
X2 = X.iloc[:, :20]
X3 = X
```

```
def evaluate_model(features):

    X_train, X_test, y_train, y_test = train_test_split(
        features,
        y,
        test_size=0.3,
        random_state=42
    )

    model = GaussianNB()

    model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)

print("Accuracy :", accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred))
print("Recall   :", recall_score(y_test, pred))
```

```
print("Feature Set 1")
evaluate_model(X1)

print("\nFeature Set 2")
evaluate_model(X2)

print("\nFeature Set 3")
evaluate_model(X3)
```

```
Feature Set 1
Accuracy : 0.9298245614035088
Precision: 0.9285714285714286
Recall   : 0.9629629629629629
```

```
Feature Set 2
Accuracy : 0.9122807017543859
Precision: 0.911504424778761
Recall   : 0.9537037037037037
```

```
Feature Set 3
Accuracy : 0.9415204678362573
Precision: 0.9454545454545454
Recall   : 0.9629629629629629
```

```
scaler = MinMaxScaler()

X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled,
    y,
    test_size=0.3,
    random_state=42
)
```

```
#Without laplace smoothing
model1 = MultinomialNB(alpha=0)

model1.fit(X_train, y_train)

pred1 = model1.predict(X_test)

print("Without Laplace")

print("Accuracy :", accuracy_score(y_test, pred1))
print("Precision:", precision_score(y_test, pred1))
print("Recall   :", recall_score(y_test, pred1))
```

```
Without Laplace
Accuracy : 0.8362573099415205
Precision: 0.7941176470588235
Recall   : 1.0
```

```
#With Laplace smoothing
model2 = MultinomialNB(alpha=1)

model2.fit(X_train, y_train)

pred2 = model2.predict(X_test)

print("\nWith Laplace")

print("Accuracy :", accuracy_score(y_test, pred2))
print("Precision:", precision_score(y_test, pred2))
print("Recall   :", recall_score(y_test, pred2))
```

```
With Laplace
Accuracy : 0.8304093567251462
```

```
Precision: 0.7883211678832117
Recall    : 1.0
```

EXPERIMENT 2: Maximum Likelihood Estimation vs Bayesian Estimation

```
import numpy as np
import matplotlib.pyplot as plt

from scipy.stats import norm
```

```
np.random.seed(10)

data = np.random.normal(
    loc=50,
    scale=10,
    size=200
)
```

```
mle_mean = np.mean(data)

mle_variance = np.var(data)

print("MLE Mean =", mle_mean)

print("MLE Variance =", mle_variance)
```

```
MLE Mean = 50.743069233166835
MLE Variance = 95.30511976376441
```

```
prior_mean = 45

prior_variance = 25

sample_variance = np.var(data)

n = len(data)
posterior_mean = (
    (prior_mean/prior_variance) +
    (n*mle_mean/sample_variance)
) / (
    (1/prior_variance) +
    (n/sample_variance)
)

print("Posterior Mean =", posterior_mean)
```

```
Posterior Mean = 50.63564801143165
```

```
small = data[:10]

small_mean = np.mean(small)

small_var = np.var(small)

print("Small Sample Mean =", small_mean)
n = len(small)

posterior_small = (
    (prior_mean/prior_variance) +
    (n*small_mean/small_var)
) / (
    (1/prior_variance) +
    (n/small_var)
)

print("Bayesian Mean =", posterior_small)
```

```
Small Sample Mean = 50.5980830814079
Bayesian Mean = 49.56544020694722
```

```
large = data

large_mean = np.mean(large)
```

```
large_var = np.var(large)

n = len(large)

posterior_large = (
    (prior_mean/prior_variance) +
    (n*large_mean/large_var)
) / (
    (1/prior_variance) +
    (n/large_var)
)

print("Large Sample Bayesian Mean =", posterior_large)
```

```
Large Sample Bayesian Mean = 50.63564801143165
```

```
labels = ["MLE", "Bayesian Small", "Bayesian Large"]

values = [
    mle_mean,
    posterior_small,
    posterior_large
]

plt.figure(figsize=(6,4))
plt.bar(labels, values)
plt.ylabel("Estimated Mean")
plt.title("MLE vs Bayesian Estimation")
plt.show()
```

